CSCI-C 212 Practice Exam 2 Spring 2026 (80 points)
Mar 30, 2026

C212 Exam 2 Rubric

1. (20 points) Design the `Set<String> uniqueShortestWords(List<String> words, int k)` method that, when given a list of strings and an integer $k \geq 1$, returns a set of size at most k of the strings that contain the fewest characters. If two strings contain the same number of characters, e.g., `"dog"` and `"cat"`, then prioritize the string that comes first lexicographically.

   Hint: Use a `PriorityQueue` with a custom `Comparator`!

   Solution.

```java
/**
 * Returns at most k shortest words from the given list. If two words have the
 * same length then the one that comes lexicographically first is prioritzed.
 * @param words list of words.
 * @param k integer >= 1.
 * @return list of length at most k.
 */
Set<String> uniqueShortestWords(List<String> words, int k) {
  Set<String> shortest = new LinkedHashSet<>();
  Queue<String> pq = new PriorityQueue<>(cmp());
  for (String s : words) {
    pq.add(s);
  }
  while (k > 0 && !pq.isEmpty()) {
    String word = pq.poll();
    if (!shortest.contains(word)) {
      shortest.add(word);
      k--;
    }
  }
  return shortest;
}

private static Comparator<String> cmp() {
  return new Comparator<>() {
    @Override
    public int compare(String s1, String s2) {
      if (s1.length() < s2.length()) {
        return -1;
      } else if (s1.length() > s2.length()) {
        return 1;
      } else {
        return s1.compareTo(s2);
      }
    }
  };
}
```

2. (30 points) This question has two parts, each of which are weighted equally.

   Two integers in a list of integers are *valley-contributing* if the integer between them is strictly greater than both. For example, in the list $[3, 5, 18, 4, 1, 2]$, 5 and 4 are valley-contributing because the number in between, namely 18, is larger than both 5 and 4.

   (a) *(15 points)* Design the *tail recursive* `Optional<Integer> countValleyContributingTR(List<Integer> ls)` method that counts the number of valley-contributing integers.

   Using the above list as an example, we would return an `Optional` containing `2`. If there are fewer than 3 values in the given list, return an empty `Optional`. You will need to design a helper method. Remember the relevant access modifiers!

   Hint: We don't require that your helper method return an `Optional`; only that your driver does. So, as long as your helper is tail recursive, you can process its return value however you'd like in the driver, e.g., with a case analysis.

   Your helper method must be tail recursive or you will receive no credit.

```
static Optional<Integer> countValleyContributingTR(List<Integer> ls) {
  if (ls.size() < 3) {
    return Optional.empty();
  } else {
    return Optional.of(cVCTRHelper(ls, 2, 0));
  }
}

private static int cVCTRHelper(List<Integer> ls, int i, int acc) {
  if (i >= ls.size()) {
    return acc;
  } else {
    int fst = ls.get(i - 2);
    int mid = ls.get(i - 1);
    int lst = ls.get(i);
    if (mid > fst && mid > lst) {
      return cVCTRHelper(ls, i + 1, acc + 2);
    } else {
      return cVCTRHelper(ls, i + 1, acc);
    }
  }
}
```

(b) *(15 points)* Design the *iterative* `Optional<Integer> countValleyContributingLoop(List<Integer> ls)` method that solves the same problem as (a), but uses a loop.

Your method must use a loop and not be recursive or you will receive no credit.

```
static Optional<Integer> countValleyContributingLoop(List<Integer> ls) {
  if (ls.size() < 3) {
    return Optional.empty();
  } else {
    int acc = 0;
    int i = 2;
    while (!(i >= ls.size())) {
      int fst = ls.get(i - 2);
      int mid = ls.get(i - 1);
      int lst = ls.get(i);
      if (mid > fst && mid > lst) {
        i = i + 1;
        acc = acc + 2;
      } else {
        i = i + 1;
      }
    }
    return Opional.of(acc);
  }
}
```

3. (30 points) A *range* describes an interval between two types of ordered objects. For example, the range from 3 to 67 describes a range where the minimum is 3 and the maximum is 67. We trivially know how to compare integers, but we can extend the idea of a range to any type that can be compared. In this question, you will implement a generic class called `Range` to represent an interval.

   (a) *(5 points)* Fill in the `Range` class fields and constructor. The constructor receives three values: `T min`, `T max`, and `Comparator<T> cmp`. The first and second parameters designate the minimum and maximum values of the `Range`, and `cmp` denotes how to compare objects of type `T`. Store these three parameters as `final` instance variables. You may assume that `min` is always strictly less than `max` according to `cmp`.

```
class Range<T> {

  private final T MIN;
  private final T MAX;
  private final Comparator<T> CMP;

  // Constructor.
  Range(T min, T max, Comparator<T> cmp) {
    this.MIN = min;
    this.MAX = max;
    this.CMP = cmp;
  }
}
```

(b) *(5 points)* Design the `boolean strictlyContains(T v)` method, which returns whether v is strictly contained in `this` range. A value v is strictly contained within the `Range` if `min` $< v <$ `max`.

Hint: `Comparator<T>` objects have the `compare` method that you can invoke on the object. Pass to it two values t_1 and t_2 of type `T`, and it returns a negative integer if $t_1 < t_2$, a positive integer if $t_1 > t_2$, and 0 if t_1 is equal to t_2.

```
/**
 * Determines whether a given value is contained in this Range. A value is
 * in this Range if this.MIN.compareTo(v) < 0 and this.MAX.compareTo(v) > 0.
 * @param v value to check.
 * @return true if v is in this Range; false otherwise.
 */
boolean strictlyContains(T v) {
  return this.CMP.compare(v, this.MIN) > 0
      && this.CMP.compare(v, this.MAX) < 0;
}
```

(c) *(10 points)* Design the `boolean overlapsWith(Range<T> othRange)` method that returns whether `this` range overlaps with the given `Range`. Note that this should be a strict overlap and not simply "touching." For example, a range whose minimum is 3 and maximum is 67 does not overlap with a range whose minimum is 67 and whose maximum is 71.

```
/**
 * Checks to see if the given Range overlaps with this Range, using a strict
 * overlap criteria. That is, two Ranges overlap if they are actually over-
 * lapping and not just touching.
 * @param othRange other range to check.
 * @return true if this Range overlaps with the given Range; false otherwise.
 */
boolean overlapsWith(Range<T> othRange) {
  return this.CMP.compare(this.MIN, othRange.MAX) < 0
      && this.CMP.compare(othRange.MIN, this.MAX) < 0;
}
```

(d) *(5 points)* Override the `equals` method, which returns whether this `Range` is equal to another object. Two `Range` instances are equal if and only if they have equal minimum and maximum values.

```
@Override
public boolean equals(Object o) {
  if (!(o instanceof Range<?>)) {
    return false;
  } else {
    Range<?> r = (Range<?>) o;
    return this.MIN.equals(r.MIN) && this.MAX.equals(r.MAX);
  }
}
```

(e) *(5 points)* Override the `hashCode` method, which returns the hash code of this `Range`. The hash code is determined by invoking `Objects.hash` on the minimum and maximum values.

```
@Override
public int hashCode() {
  return Objects.hash(this.MIN, this.MAX);
}
```